

Python基礎知識ガイド

プログラミング言語Pythonにおいて、最低限の基礎知識を解説するドキュメントです。
さらに詳しく知りたい・理解したいという方は「Python 3 公式ドキュメント」を参照してください。
詳細な仕様とコーディング例を確認できます。
Pythonは海外発祥のプログラミング言語になりますが、公式ドキュメントは日本語に翻訳されているため読みやすく設計されています。

▼Python 3公式ドキュメント▼

<https://docs.python.org/ja/3/>

<目次>

1. Pythonの実行方法
 - 1.1. インタープリタの起動
 - 1.2. pythonファイルの実行
2. Pythonプログラムの基本構造
3. 基本データ型
 - 3.1. 文字列データ
 - 3.1.1. 文字列の生成
 - 3.1.2. 文字列の型変換
 - 3.1.3. 文字列の連結と繰り返し
 - 3.1.4. 文字列の抽出
 - 3.1.5. 文字列の書式設定
 - 3.2. 数値データ
 - 3.2.1. 数値への型変換
4. コレクション
 - 4.1. リスト
 - 4.1.1. リストの生成
 - 4.1.2. リストへの型変換
 - 4.1.3. 要素の抽出
 - 4.1.4. 要素の書き換え
 - 4.2. タプル
 - 4.2.1. タプルの生成
 - 4.2.2. タプルへの型変換
 - 4.2.3. 要素の抽出
 - 4.3. 辞書
 - 4.3.1. 辞書の生成
 - 4.3.2. 要素の抽出
 - 4.3.3. 要素の追加と書き換え
5. 条件分岐
6. ループ
 - 6.1. for文
 - 6.2. while文
7. 関数
8. クラス
9. モジュールとパッケージ

1. Pythonの実行方法

ここでは、2種類のPython実行方法を解説します。

実行方法	概要
インタプリタの起動	実行結果を1つずつ確認しながらプログラムを実行
Pythonファイルの実行	プログラムを書いたファイルを指定して実行

1.1. インタプリタの起動

簡易的なプログラムを試す場合は、インタプリタといった方法でPythonを実行するのが便利です。Pythonのプログラムの実行結果を1つずつ確認しながらプログラムを実行できるのが特徴になります。インタプリタを起動するには、VSCodeのターミナルで以下のようにコマンドを実行してください。Windowsならpythonコマンド、macOSならpython3コマンドを実行してください。「>>>」が表示されればインタプリタが起動しています。

▼Windowsの場合

```
> python
>>>
```

▼macOSの場合

```
% python3
>>>
```

インタプリタを終了するには、「exit()」とコマンドを入力してからEnterキーを押してください。

1.2. Pythonファイルの実行

一定の長さを持つプログラムや何度も実行するようなプログラムは、インタプリタによる実行は向いていません。そこでプログラムを書いたファイルを指定して実行する方法があります。Windowsの場合は、VSCodeのターミナル上で「python <ファイル名>」、macOSの場合は「python3 <ファイル名>」と入力してプログラムファイルを実行します。

▼Windowsの場合

```
> python sample.py
```

▼macOSの場合

```
% python3 sample.py
```

ディレクトリ以外の場所で実行する場合は、パス付きでファイル名を指定してください。

2. Pythonプログラムの基本構造

Pythonには、基本的な構文ルールが存在します。

以下は、Pythonプログラムを記述する際に「必ず守るべき」構造上のルールです。

ルール	説明	ポイント
インデント	ブロックの内部は半角スペース4つで字下げ	スペースとタブを混ぜないこと
コロン(:)	制御構文の行末に付ける	コロンの後は必ずインデントが続く
文の終わり	セミコロンは不要. 1行に複数文書くときのみ使用可能	原則1行に1文が推奨
コメント	コメントは#から始める. 文末にも書ける	説明が必要な箇所には積極的に使うこと
関数定義	def 関数名(引数): で始め 内部をインデントで構成	定義と処理ブロックは一続きにする
クラス定義	class クラス名: で始め 内部に__init__などの関数を定義	インデントの入れ子に注意
空行	複数の関数やクラスの間には空行を1~2行入れる	コードの可読性が大きく向上する
変数名/定数名	通常はスネークケース(snake_case)を使う	クラスはキャメルケース(CamelCase)

▼ 実例

```
def greet(name): # 関数定義とコメント
    if name: # 条件文とコロン
        print(f"Hello, {name}") # インデントされた処理
    else:
        print("Hello, stranger")
```

▼ Python構文のまとめ

- ・インデントは構文の一部である(4スペースが標準)
- ・コロン(:)が付く行の後には必ずインデントされたブロックが続く
- ・波かっこは使わない(C言語系と違う点)
- ・コメントや空行で可読性を高める
- ・関数・変数の命名規則も大事(snake_case vs CamelCase)

さらに詳しく学びたい場合は、PEP 8(Pythonの公式スタイルガイド)もおすすめです。

<https://pep8-ja.readthedocs.io/ja/latest/>

3. 基本データ型

Pythonに限らず、プログラムを扱う際に以下の基本データ型を理解する必要があります。

基本データ型	概要
文字列データ	" , ' "によって括られたデータは文字列型として認識される
数値データ	整数 (int) , 小数 (float) として認識される

3.1. 文字列データ

ここでは、基本的な文字列データの扱い方を解説します。

文字列データの扱い方	概要
文字列の生成	文字列リテラル (例: "Hello") を使って新しい文字列を作成
文字列への型変換	<code>str()</code> 関数を使って数値や他の型を文字列に変換
文字列の連結と繰り返し	<code>+</code> 演算子で文字列を結合し、 <code>*</code> 演算子で指定回数だけ繰り返す
文字列の抽出	インデックス (例: <code>s[0]</code>) やスライス (例: <code>s[1:4]</code>) で部分文字列を取得
文字列の書式設定	<code>f</code> 文字列 (例: <code>f"{name}さん"</code>) や <code>format()</code> メソッドを使用

3.1.1. 文字列の生成

シングルクォーテーション `' '` またはダブルクォーテーション `" "` で括られた文字列は文字列型として認識されます。

Python 3系はUnicodeを標準でサポートしているので日本語も使えます。

▼ 実例

```
moji = 'Python'
kazu = "123"
```

数値に関しても、シングルあるいはダブルクォーテーションで括ることによって文字列データと認識されます。

3.1.2. 文字列の型変換

`str`関数を使うことで他の型から文字列型に変換できます。

次の例では数値を文字列型に変換しています。

▼実例

```
>>> str(10)
'10'
```

3.1.3. 文字列の連結と繰り返し

「+」演算子を使うことで文字列を連結し1つの文字列にすることができます。

▼実例

```
>>> 'あいうえお' + 'かきくけこ'
'あいうえおかきくけこ'
```

「*」演算子で文字列を繰り返すこともできます。

▼実例

```
>>> 'あいうえお' * 2
'あいうえおあいうえお'
```

3.1.4. 文字列の抽出

文字列の後ろに角括弧([])を付けることで文字列内の文字を抽出できます。

1文字だけ抽出したい場合は、左から0、1、... と数えて何番目の文字を抽出するかを角括弧内に書きます。右から数える場合は、-1、-2、... と数えます。

▼実例

```
>>> 'あいうえお'[1]
'い'
>>> 'あいうえお'[-1]
'お'
```

複数の文字を抽出する場合は、[開始位置 : 終了位置 : ステップ数]を指定します。

開始位置は1文字の抽出のときと同様に数えます。終了位置も数え方は一緒ですが、抽出されるのは指定した位置より1つ手前の文字までなので注意が必要です。

ステップ数で指定数だけスキップしながら文字を抽出することもできます。

▼実例

```
>>> 'あいうえお'[:]
'あいうえお'
>>> 'あいうえお'[:2]
'あい'
>>> 'あいうえお'[2:]
'うえお'
>>> 'あいうえお'[2:4]
'うえ'
```

```
>>> 'あいうえお'[:2]
'あう'
>>> 'あいうえお'[1:4:2]
'いえ'
```

3.1.5. 文字列の書式設定

formatメソッドを使って値を文字列に埋め込むことができます。

具体的な用途としては、メールの書式(format)を決めておいて値を動的に埋め込むときなどに使えます。以下の例では、文字「Job」と「Code」を埋め込んでから出力します。

Python 3.6以降では、文字列の中に値を直接埋め込むことができる「f-string」という方法も使えます。

本ガイドでは利用しないため説明は割愛しますが、詳しく知りたい方はPython公式サイトをご覧ください。

▼ 実例

```
>>> a='あ'
>>> b='い'
>>> '{} {}'.format(a, b)
'あ い'
>>> '{1} {0}'.format(a, b)
'い あ'
>>> '{arg1} {arg2}'.format(arg1=b, arg2=a)
'い あ'
```

▼ Python公式サイト:フォーマット済み文字列リテラル(f-string) ▼

<https://docs.python.org/ja/3.11/tutorial/inputoutput.html#formatted-string-literals>

3.2. 数値データ

ここでは、基本的な数値データの扱い方を解説します。

数値データの扱い方	概要
数値への型変換	数値データをint↔floatへ型変換する

3.2.1. 数値への型変換

文字列の時と同様にint()関数とfloat()関数を使うことで、int及びfloatに他の型から型変換することができます。

以下は文字列からintまたはfloat型へ型変換した例です。

▼実例

```
>>> int('10')
10
>>> float('10.2')
10.2
```

Python3の整数と小数はそれぞれ intとfloatという型で扱います。

「10」や「10.2」とすれば前者はint、後者はfloatと自動で解釈してくれます。

算術演算では表の算術演算子を使うことができます。

▼算術演算子

演算子	用途	例
+	加算	>>> 4 + 2 6
-	減算	>>> 4 - 2 2
*	乗算	>>> 4 * 2 8
/	除算(計算結果はfloat型)	>>> 4 / 2 2.0
//	除算(計算結果はint型, 小数点以下切り捨て)	>>> 4 // 2 2
%	剰余	>>> 4 % 2 0
**	累乗	>>> 4 ** 2 16

4. コレクション

コレクションとは、データをまとめて管理できるデータ型のことです。
ここでは、基本的なコレクションの扱い方を解説します。

▼よく利用されるコレクション

コレクション名	概要
リスト(list)	順序があるデータの並びを表すコレクション
タプル(tuple)	リスト同様、順序があるデータの並びを表すコレクション
辞書(dictionary)	キーと値をセットにしてデータを管理するコレクション

各コレクションの扱い方を理解していきましょう。

4.1. リスト

ここでは、基本的なリストデータの扱い方を解説します。

リストデータの扱い方	概要
リストの生成	角括弧([])内にカンマ(,)でリスト生成
リストへの型変換	他の型からリストに型変換
要素の抽出	角括弧([])を使って要素抽出
要素の書き換え	角括弧([])を使って要素指定

4.1.1. リストの生成

角括弧([])内にカンマ(,)区切りでデータを並べるとリストを生成できます。

▼実例

```
>>> ['あ', 'い', 'う', 'え', 'お']  
['あ', 'い', 'う', 'え', 'お']
```

4.1.2. リストへの型変換

list関数を使えば、他の型からリストに型変換できます。
次の例では文字列をリストに変換しています。

▼実例

```
>>> list('あいうえお')  
['あ', 'い', 'う', 'え', 'お']
```


4.1.3. 要素の抽出

リストは角括弧([])を使って要素を抽出できます。
特定のリストデータを抽出したい場合は、左から0、1、... と数えて何番目の文字を抽出するかを角括弧内に書きます。右から数える場合は、-1、-2、... と数えます。
取り出し方は前述した「文字列の抽出」と同様です。

▼実例

```
>>> a = ['あ', 'い', 'う', 'え', 'お']
>>> a[1]
'い'
>>> a[-1]
'お'
>>> a[1:4:2]
['い', 'え']
```

4.1.4. 要素の書き換え

角括弧([])を使って要素を指定して値を書き換えることができます。

▼実例

```
>>> a = ['あ', 'い', 'う', 'え', 'お']
>>> a[4] = 'ん'
>>> a
['あ', 'い', 'う', 'え', 'ん']
```

4.2. タプル

ここでは、基本的なタプルデータの扱い方を解説します。

タプルデータの扱い方	概要
タプルの生成	角括弧([])内にカンマ(,)でリスト生成
タプルへの型変換	他の型からリストに型変換
要素の抽出	角括弧([])を使って要素抽出

タプルはリストと同じく、順序があるデータの並びを表すコレクションです。
ただし、タプルは要素を書き換えることができません。
定数の並びを表現するときなどに使えます。

4.2.1. タプルの生成

タプルはカンマ(,)区切りでデータを並べるだけで生成できます。

さらに、丸括弧「()」を付けても同じようにタプルを生成できます。
丸括弧があったほうが紛らわしくないので付けておくことを推奨します。

▼ 実例

```
>>> ('あ', 'い', 'う', 'え', 'お')
('あ', 'い', 'う', 'え', 'お')
```

4.2.2. タプルへの型変換

tuple関数を使ってリストをタプルに変換することができます。

▼ 実例

```
>>> tuple(['あ', 'い', 'う', 'え', 'お'])
('あ', 'い', 'う', 'え', 'お')
```

4.2.3. 要素の抽出

リストと同じように角括弧「[]」を使って要素を抽出することができます。

▼ 実例

```
>>> a = ('あ', 'い', 'う', 'え', 'お')
>>> a[1]
'い'
>>> a[-1]
'お'
>>> a[1:4:2]
('い', 'え')
```

4.3. 辞書

ここでは、基本的な辞書データの扱い方を解説します。

辞書データの扱い方	概要
辞書の生成	波括弧({ })の中に「キー: 値」のセットをカンマ(,)区切りで辞書生成
要素の抽出	getメソッドでキーを指定して値取得
要素の追加と書き換え	存在しないキーを指定して要素追加 存在するキーを指定して値書き換え

辞書はキーと値をセットにしてデータを管理するコレクションです。
リストやタプルと違ってデータに順序はなく、データの書き換えが可能なコレクションとなります。

4.3.1. 辞書の生成

波括弧({ })の中に「キー: 値」のセットをカンマ(,)区切りで並べると辞書を生成できます。

▼実例

```
>>> {'a': 'あ', 'b': 'い', 'c': 'う', 'd': 'え', 'e': 'お'}  
{'a': 'あ', 'b': 'い', 'c': 'う', 'd': 'え', 'e': 'お'}
```

4.3.2. 要素の抽出

要素は角括弧([])かgetメソッドでキーを指定すれば値を取得できます。

角括弧を使った場合、存在しないキーを指定するとエラーになります。

getメソッドを使う場合は、第2引数にキーが存在しない場合の出力値を設定できます。

第2引数を指定しなくてもエラーになるのではなくNoneが返ってきます。(画面上には何も表示されません)

▼実例

```
>>> data = {'a': 'あ', 'b': 'い', 'c': 'う', 'd': 'え', 'e': 'お'}  
>>> data['b']  
'い'  
>>> data.get('b')  
'い'  
>>> data['f']  
Traceback (most recent call last):  
File "<stdin>", line 1, in <module>  
KeyError: 'f'  
>>> data.get('f', 'nothing')  
'nothing'  
>>> data.get('f')  
>>>
```

4.3.3. 要素の追加と書き換え

存在しないキーを指定して代入操作をすると、要素を追加できます。

存在するキーを指定して代入操作をすると、値の書き換えができます。

▼実例

```
>>> data = {'a': 'あ', 'b': 'い', 'c': 'う', 'd': 'え', 'e': 'お'}  
>>> data['f'] = 'か'  
>>> data  
{'a': 'あ', 'b': 'い', 'c': 'う', 'd': 'え', 'e': 'お', 'f': 'か'}  
>>> data['f'] = 'ん'  
>>> data  
{'a': 'あ', 'b': 'い', 'c': 'う', 'd': 'え', 'e': 'お', 'f': 'ん'}
```

5. 条件分岐

ifやelifのあとにTrue(真)またはFalse(偽)となる式や値を記述すると処理を分岐できます。

ifやelifの文末にコロン(:)が必要です。

実例では、xが'Job'でも'Code'でもなければelseのprint関数が実行されます。

==は比較演算子の一種で、比較演算子にはその他に表のようなものがあります。

また、論理演算子も分岐条件の判定に用いられます。

▼実例

```
if x == 'Job':  
    print('Job')  
elif x == 'Code':  
    print('Code')  
else:  
    print('not Job Code')
```

▼比較演算子

比較演算子	意味	使用例
==	等号	>>> 'あ' == 'あ' True
!=	等号否定	>>> 'あ' != 'あ' False
>, >=, <, <=	不等号	>>> 1 < 5 True
in	要素に存在するか	>>> 'あ' in ['あ', 'い', 'う'] True

▼論理演算子

論理演算子	意味	使用例
and	論理積	>>> True and False False
or	論理和	>>> True or False True
not	否定	>>> not True False

6. ループ

ここでは、基本的なループの扱い方を解説します。

▼ループの種類

ループの種類	概要
for文	指定回数などによるループ
while文	条件による無限ループ

6.1. for文

実例では、forを使ったループの例を示しており実行結果も載せています。
range関数は数値のシーケンスを作り出すことができ、使い方は以下の通りです。

▼range関数とは

range(開始数値, 終了数値, スキップ数)	・終了数値のみ必須 ・開始数値を省略した場合は0から開始 ・スキップ数を2とすると1の次は3となる ・終了数値1つ前の数値が取り出される
--------------------------	--

実例では、1から5までの数値が順番に変数iに格納されループします。このように「for ... in ...」という書き方はPythonの書き方でよく使われます。
range関数の他にもリストやタプルや辞書などでもループを回せます。また、辞書の場合はキーが取り出されることになります。breakとcontinueはループ制御に使われていますが、breakはループから抜け出す命令でcontinueは次のループ処理に移る命令になります。

▼実例

```
for i in range(1, 6):
    if i == 2:
        print(i, ' if')
    elif i == 4:
        print(i, ' elif')
        break
    else:
        print(i, ' else')
        continue
```

上記のプログラムを実行すると次の実行結果になります。

▼実行結果

```
1 else
2 if
3 else
4 elif
```

6.2. while文

whileを使ったループも記述することができます。

whileは後ろに書いた条件が満たされる限りループするため「while True:」と書いて無限ループを作ることができます。

以下の例では「i += 1」と書いてありますが、これは変数iを1増やす命令です。

他のプログラミング言語に見られる「i++」という書き方はできないので注意してください。

▼実例

```
i = 1
while True:
    print(i)
    i += 1
    if i == 5:
        break
```

上記のプログラムを実行すると次の実行結果になります。

▼実行結果

```
1
2
3
4
```

7. 関数

Pythonの関数定義は「def」からはじまり、半角スペースを入れてから関数名を書きます。

▼関数の記述方法

```
def <関数名>():
```

使用例での関数定義の引数では「arg3='baz'」としていますが、これは関数呼び出しで arg3 を指定しなくてもデフォルトで arg3 の引数に「'baz'」を設定してくれる書き方となっています。関数呼び出しでは引数名を指定(arg2='bar')することができますが(キーワード引数と呼びます)、引数名なしの指定(使用例では「'foo'」。位置引数と呼びます)をすべて書き終わったあとに書かなければならない点に注意が必要です。

▼関数の使用例

```
# 関数定義
def func_name(arg1, arg2, arg3='baz'):
    print(arg1, arg2, arg3)

# 関数呼び出し
func_name('foo', 'bar') # foo bar baz
func_name('foo', arg2='bar', arg3='qux') # foo bar qux
```

引数の書き方で「*<引数名>」と「**<引数名>」はDjangoでもよく見られる形です。前者は位置引数であふれた引数を1つのタプルにまとめる働きをします。後者はキーワード引数を1つの辞書にまとめる働きをします。なお、引数名は「args」と「kwargs」という名前だけでなくてもよいのですが、慣例でこの名前がよく使われます。

▼関数(*args, **kwargs)の使用例

```
# 関数定義
def func_name(arg, *args, **kwargs):
    print(arg, args, kwargs)

# 関数呼び出し
func_name('foo', 'bar', 'baz', a='qux', b='quux') # foo ('bar', 'baz')
{'a': 'qux', 'b': 'quux'}
```

8. クラス

クラスを定義するには以下のように書きます。
丸括弧「()」の中に親クラスを書いて継承を定義することもできます。

▼クラスの記述方法

```
class <クラス名>():
```

クラス変数は①のように「<クラス変数名> =」で定義でき、インスタンス変数は通常__init__メソッド(コンストラクタ)の中で③のように「self.<インスタンス変数名> =」で定義します。
クラス変数とインスタンス変数の呼び出しはそれぞれ「<クラス名>.<クラス変数名>」(②)、「self.<インスタンス変数名>」(④)のように行います。メソッドは「def <メソッド名>(self):」で定義できます。第1引数は必ず自身のインスタンスを表すselfと決まっています。ほかの引数はselfに続けてカンマ(,)区切りで記述します。

例えば「def <メソッド名>(self, <引数1>, <引数2>)」のように書きます。

親クラスのメソッドは子クラスでオーバーライドできますが、オーバーライドした子クラスのメソッドを呼び出しても親クラスのメソッドは見えなくなってしまう(⑥)。

なお、子クラスから親クラスのメソッドを使いたい場合は、super()を使って呼び出すことができます(⑤)。

▼クラスの使用例

```
# 親クラス
class Parent():
    # クラス変数の設定
    var_class = 'parent' ①

    # 親子共通メソッド
    def method_common(self):
        print('method_common ' + Parent.var_class) ②

    # 親クラスの独自メソッド
    def method_parent(self):
        print('method ' + Parent.var_class)

# 子クラス
class Child(Parent):
    # コンストラクタ
    def __init__(self, value):
        # インスタンス変数の設定
        self.var_instance = value ③

    # 親子共通メソッドのオーバーライド
    def method_common(self):
        print('method_common ' + self.var_instance) ④

    # 子クラスの独自メソッド
```



```

def method_child(self):
    print('method ' + self.var_instance)
    super().method_parent() ⑤

if __name__ == '__main__':
    # 子クラスのインスタンス化
    child = Child('child')

    # オーバーライドしたメソッドを呼び出して出力値を確認
    child.method_common() # 出力 => method_common child ⑥

    # 子クラスの独自メソッドから親クラスのメソッドを呼び出して出力値を確認
    child.method_child() # 出力 => method_child / method_parent

    # 子クラスには定義していない親クラスのメソッドを呼び出して出力値を確認
    child.method_parent() # 出力 => method_parent

```

▼コンストラクタとは

コンストラクタは、オブジェクトが生成される時に実行されるメソッド(関数)
オブジェクトが扱う変数などの初期化を行います

▼オーバーライドとは

「親クラス」で定義されたインスタンスメソッドを継承した「子クラス」で
独自に定義し直して上書きすることを指します
「子クラス」で機能を追加する必要がある場合などにオーバーライドを利用します

9. モジュールとパッケージ

関数やクラスをまとめたPythonファイル(拡張子は.py)のことを「モジュール」と呼びます。そして、このモジュールを束ねて__init__.pyファイルを含んだディレクトリをパッケージと呼びます。モジュールは他のパッケージにあるモジュールを読み込んで利用することができます。例えば、図のような構成になっているとします。

▼モジュールとパッケージの構成例

```
|— import_test.py
|— package
    |— __init__.py
    |— module.py
```

import_test.pyからmodule.pyに定義した関数(func)を利用するには、次の3通りの方法があります。

1. **import** <パッケージ名>.<モジュール名>
2. **from** <パッケージ名> **import** <モジュール名>
3. **from** <パッケージ名>.<モジュール名> **import** <オブジェクト名>(<関数名など >)

▼他パッケージにあるモジュール内の関数を利用する方法 - 1

```
import package.module

package.module.func() #関数を呼び出し
```

▼他パッケージにあるモジュール内の関数を利用する方法 - 2

```
from package import module

module.func() #関数を呼び出し
```

▼他パッケージにあるモジュール内の関数を利用する方法 - 3

```
from package.module import func

func() #関数を呼び出し
```

それぞれの例で関数の呼び出し方が違う点に注意しましょう。